

PySpark Code for CNN Model - Crack Detection

(Muneendra Magani, Madhumitha Mandyam, **Nandhika Rajmanikandan**, Ganesb Vannam)

Code for Simple CNN Model:

Required Packages:

```
import os
import tensorflow as tf
import numpy as np
from pyspark.sql import SparkSession
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

Source Code:

```
spark = SparkSession.builder.appName("Surface Crack Detection").config("spark.executor.memory",
"4g").getOrCreate()
```

```
cracked_dir = os.path.join(path, 'Positive')
non_cracked_dir = os.path.join(path, 'Negative')
```

```
def load_images_from_folder(folder, label):
    images = []
    labels = []
    for filename in os.listdir(folder):
        img_path = os.path.join(folder, filename)
        img = tf.keras.utils.load_img(img_path, target_size=(64, 64)) # Resize images to 64x64
        img_array = tf.keras.utils.img_to_array(img) / 255.0 # Normalize pixel values
        images.append(img_array)
        labels.append(label)
    return images, labels
```

```
cracked_images, cracked_labels = load_images_from_folder(cracked_dir, 1)
non_cracked_images, non_cracked_labels = load_images_from_folder(non_cracked_dir, 0)
```

```
images = np.array(cracked_images + non_cracked_images)
labels = np.array(cracked_labels + non_cracked_labels)
```

```
labels = to_categorical(labels, num_classes=2)
```

```
X_train, X_temp, y_train, y_temp = train_test_split(images, labels, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

```
datagen = ImageDataGenerator(rotation_range=15, width_shift_range=0.1, height_shift_range=0.1,
horizontal_flip=True)
datagen.fit(X_train)
```

```
def create_qcnn():
    inputs = Input(shape=(64, 64, 3))
```

```
    x = Conv2D(32, (3, 3), activation="relu", padding="same")(inputs)
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(2, 2))(x)
```

```
    x = Conv2D(64, (3, 3), activation="relu", padding="same")(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(2, 2))(x)
```

```
    x = Flatten()(x)
    x = Dense(128, activation="relu")(x)
    x = Dropout(0.5)(x)
```

```
    x = Dense(64, activation="relu")(x)
    x = Dropout(0.5)(x)
```

```

outputs = Dense(2, activation="softmax")(x)

model = Model(inputs=inputs, outputs=outputs)
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
return model

model = create_qcnn()
model.summary()

history = model.fit(datagen.flow(X_train, y_train, batch_size=64),
                    validation_data=(X_val, y_val),
                    epochs=4,
                    verbose=1)

test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=1)
print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_accuracy:.4f}")

y_pred = np.argmax(model.predict(X_test), axis=1)
y_true = np.argmax(y_test, axis=1)

print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=["Non-Cracked", "Cracked"]))

conf_matrix = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=["Non-Cracked", "Cracked"],
            yticklabels=["Non-Cracked", "Cracked"])
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

```

```
plt.figure(figsize=(12, 6))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title("Training and Validation Accuracy")
plt.show()
```